

Algorithms

Lecture #41

Syed Hamed Raza

Bellman-Ford Algorithm

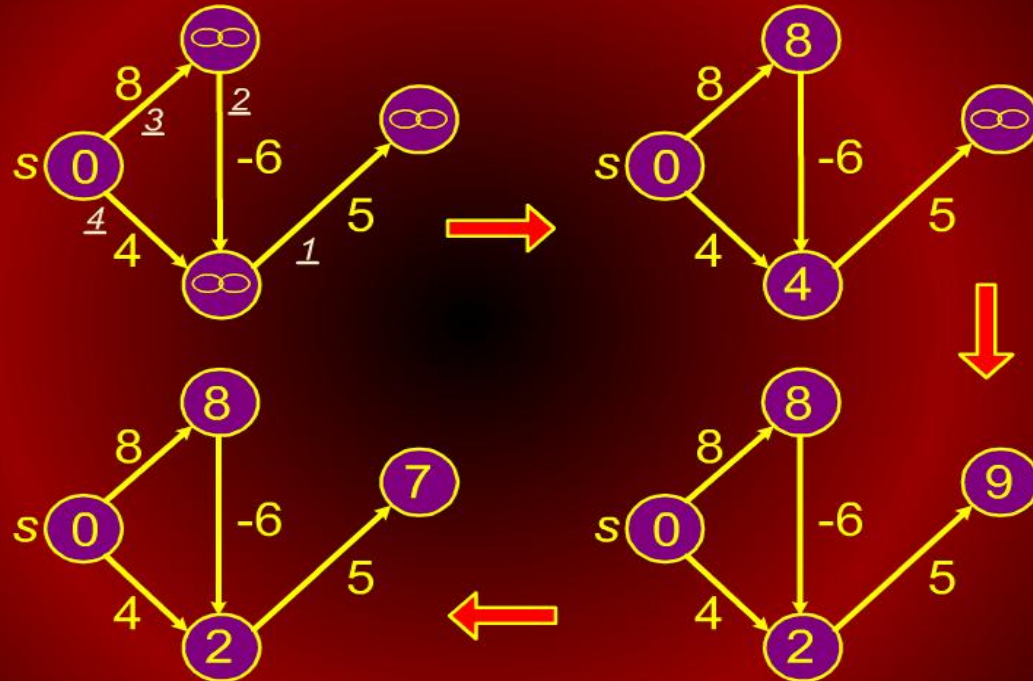
Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```
1 for ( each  $u \in V$  )  
2 do  $d[u] \leftarrow \infty$   
3    $\text{pred}[u] = \text{nil}$   
4  
5  $d[s] \leftarrow 0$ ;  
6 for  $i = 1$  to  $V - 1$   
7 do for ( each  $(u, v)$  in  $E$  )  
8   do RELAX( $u, v$ )
```

Bellman-Ford Algorithm

Bellman-Ford Algorithm



Correctness of Bellman-Ford

Correctness of Bellman-Ford

- Think of Bellman-Ford as a sort of bubble-sort analog for shortest path.
- The shortest path information is propagated sequentially along each shortest path in the graph.
- Consider any shortest path from s to some other vertex u :
 $u: \langle v_0, v_1, \dots, v_k \rangle$
where $v_0 = s$ and $v_k = u$.

Correctness of Bellman-Ford

Correctness of Bellman-Ford

- Since a shortest path will never visit the same vertex twice, we know That $k \leq V - 1$.
- Hence the path consists of at most $V - 1$ edges.
- Since this a shortest path, it is $\delta(s, v_i)$, the true shortest path cost from s to v_i that satisfies the equation:
$$\delta(s, v_i) = \delta(s, v_{i-1}) + w(v_{i-1}, v_i)$$

Correctness of Bellman-Ford

Correctness of Bellman-Ford

- We assert that after the i^{th} pass of the “for- i ” loop, $d[v_i] = \delta(s, v_i)$.
- The proof is by induction on i .
- Observe that after the initialization (pass 0), $d[v_1] = d[s] = 0$.

Correctness of Bellman-Ford

Correctness of Bellman-Ford

- In general, prior to the i^{th} pass through the loop, the induction hypothesis tells us that $d[v_{i-1}] = \delta(s, v_{i-1})$.
- After the i^{th} pass, we have done relaxation on the edge (v_{i-1}, v_i) (since we do relaxation along all edges).

Correctness of Bellman-Ford

Correctness of Bellman-Ford

- Thus after the i^{th} pass we have

$$\begin{aligned}d[v_i] &\leq d[v_{i-1}] + w(v_{i-1}, v_i) \\&= \delta(s, v_{i-1}) + w(v_{i-1}, v_i) \\&= \delta(s, v_i).\end{aligned}$$

Correctness of Bellman-Ford

Correctness of Bellman-Ford

- Recall from Dijkstra's algorithm that $d[v_i]$ is never less than $\delta(s, v_i)$.
- Thus, $d[v_i]$ is in fact equal to $\delta(s, v_i)$.
- This completes the induction proof.

Correctness of Bellman-Ford

Correctness of Bellman-Ford

- In summary, after i passes through the for loop, all vertices that are i edges away along the shortest path tree from the source have the correct values stored in $d[u]$.
- Thus, after the $(v - 1)^{\text{st}}$ iteration of the for loop, all vertices u have correct distance values stored in $d[u]$.

All-Pairs Shortest Paths

All-Pairs Shortest Paths

- We consider the generalization of the shortest path problem: to compute the shortest paths between all pairs of vertices.
- Let $G = (V, E)$ be a directed graph with edge weights.
- If $(u, v) \in E$ is an edge then $w(u, v)$ denotes its weight.

All-Pairs Shortest Paths

All-Pairs Shortest Paths

- $\delta(u, v)$ is the distance of the minimum cost path between u and v .
- We will allow G to have negative edges weights but will not allow G to have negative cost cycles.
- We will present an $\Theta(n^3)$ algorithm for the **all pairs shortest path**.
- The algorithm is called the **Floyd-Warshall algorithm** and is based on dynamic programming.

All-Pairs Shortest Paths

All-Pairs Shortest Paths

- We will use an adjacency matrix to represent the digraph.
- Because the algorithm is matrix based, we will employ the common matrix notation, using i , j and k to denote vertices rather than u , v and w .

All-Pairs Shortest Paths

All-Pairs Shortest Paths

- The input is an $n \times n$ matrix of edge weights:

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } i \neq j \text{ and } (i, j) \in E \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E \end{cases}$$

- The output will be an $n \times n$ distance matrix $D = d_{ij}$, where $d_{ij} = \delta(i, j)$, the shortest path cost from vertex i to j .

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

- The algorithm dates back to the early 60's.
- As with other dynamic programming algorithms, the genius of the algorithm is in the clever recursive formulation of the shortest path problem.
- For a path $p = \langle v_1, v_2, \dots, v_l \rangle$, we say that the vertices $v_2, v_3, \dots, v_{l-1}$ are the **intermediate vertices** of this path.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

Formulation:

- Define $d_{ij}^{(k)}$ to be the shortest path from i to j such that any intermediate vertices on the path are chosen from the set $\{1, 2, \dots, k\}$.
- The path is free to visit any subset of these vertices and in any order.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm

Bellman-Ford Algorithm

Bellman-Ford Algorithm